1 Linear Quadratic Regulator

I need to update this section, I dont think the assumptions that $S \in \mathcal{C}^1([0,T];\mathbb{S}^2)$ is enough for us to write $-H^\top S(r) + S(r)H = -2H^\top S(r)$. Even though the solution we have is correct in the end I would like to be consistent so we are not testing our code with invalid matrices.

We consider the controlled SDE

$$dX_s = (HX_s + M\alpha_s)ds + \sigma dW_s, \quad s \in [t,T], \quad X_t = x,$$

where $X_s \in \mathbb{R}^2$, $\alpha_s \in \mathbb{R}^m$, W is an $\mathbb{R}^{d'}$-valued Wiener process and $H \in \mathbb{R}^{2\times 2}$, $M \in \mathbb{R}^{2\times m}$, $\sigma \in \mathbb{R}^{2\times d'}$ are deterministic constant matrices. Our aim is to minimize, for any $(t,x) \in [0,T] \times \mathbb{R}^2$, the objective

$$J^\alpha(t,x) := \mathbb{E}^{\,t,x}\left[\int_t^T (X_s^\top C X_s + \alpha_s^\top D\alpha_s)ds + X_T^\top R X_T\right]$$

over square integrable, adapted controls processes α, where $C = C(t),\ D = D(t),\ R$ are symmetric, measurable, deterministic and bounded in t. For any $x \in \mathbb{R}^d$ and $t \in [0,T]$ we have $x^\top Cx \geq 0$ and $x^\top Rx \geq 0$. Moreover, there exists $\delta > 0$ such that for all $a \in \mathbb{R}^m$ and $t \in [0,T]$ we have $a^\top D(t)a \geq \delta$. The value function is $v(t,x) := \inf_\alpha J^\alpha(t,x)$. The associated HJB equation is

$$\partial_t v + \inf_{a\in\mathbb{R}^m}\left[\frac{1}{2}\operatorname{tr}\big(\sigma\sigma^\top \partial_x^2 v\big) + \big(x^\top H^\top + a^\top M^\top\big)\,\partial_x v + x^\top Cx + a^\top Da\right] = 0 \quad \text{on } [0,T)\times\mathbb{R}^d$$

$$v(T,x) = x^\top Rx, \quad \forall x \in \mathbb{R}^d.$$

Motivated by the quadratic terminal condition $v(T,x) = x^\top Rx$, we seek a solution to the HJB equation of the form$v(t,x) = x^\top S(t)x + b(t)$, where $S \in \mathcal{C}^1([0,T];\mathbb{S}^2)$ is a symmetric matrix-valued function and $b \in \mathcal{C}^1([0,T];\mathbb{R})$. For this choice we have that $\partial_x v(t,x) = 2S(t)x,\ \partial_x^2 v(t,x) = 2S(t)$. Substituting this ansatz into the HJB equation

$$x^\top S'(t)x + b'(t) + \operatorname{tr}(\sigma\sigma^\top S(t)) + x^\top\big(H^\top S(t) + S(t)H\big)x + x^\top Cx + \inf_{a\in\mathbb{R}^m}\big[2a^\top M^\top S(t)x + a^\top Da\big] = 0.$$

For fixed (t,x), the expression inside the infimum is a strictly convex quadratic function of a. The first–order condition is therefore

$$0 = 2M^\top S(t)x + 2Da,$$

which yields the optimal control in feedback form

$$a^*(t,x) = -D^{-1}M^\top S(t)x.$$

Substituting a^* back into the infimum gives

$$\inf_{a\in\mathbb{R}^m}\big[2a^\top M^\top S(t)x + a^\top Da\big] = -x^\top S(t)MD^{-1}M^\top S(t)x.$$

Collecting terms, we thus obtain ODEs for $S = S(t)$ and $b = b(t)$ from the HJB equation

$$S'(t) = -\big(H^\top S(t) + S(t)H\big) + S(t)MD^{-1}M^\top S(t) - C, \quad S(T) = R$$

and

$$b'(t) = -\operatorname{tr}\big(\sigma\sigma^\top S(t)\big), \quad b(T) = 0.$$

Where the first equation, the so-called Riccati ODE, admits a unique symmetric non–negative solution under the stated assumptions. Integrating the equation for b yields

$$b(t) = \int_t^T \operatorname{tr}\big(\sigma\sigma^\top S(r)\big)\ dr.$$

We therefore conclude that the value function is

$$v(t,x) = x^\top S(t)x + \int_t^T \operatorname{tr}\big(\sigma\sigma^\top S(r)\big)\ dr,$$

and the optimal Markov control is given by

$$\alpha^*(t,x) = -D^{-1}M^\top S(t)x.$$

One can convince themselves of this using the verification theorem.

1.1 LQR Solver

The main challenge in evaluating $v(t, x)$ is solving the Riccati ODE

$$S'(r) = -\big(H^\top S(r) + S(r)H\big) + S(r)MD^{-1}M^\top S(r) - C, \quad S(T) = R.$$

This is a terminal value problem, which is slightly awkward to solve directly since most ODE solvers (including `scipy.integrate.solve_ivp`) expect an initial value problem. To deal with this, we make the substitution $\tau = T - t$, which flips the direction of integration. Under this change of variables the ODE becomes

$$\frac{dS}{d\tau} = 2H^\top S - SMD^{-1}M^\top S + C, \quad S(\tau = 0) = R,$$

which is now an initial value problem we can pass directly to `solve_ivp`. We use the `RK45` method with tight tolerances (rtol $= 10^{-8}$, atol $= 10^{-10}$) to keep the Riccati solve accurate enough that it doesn't become the bottleneck when we later do Monte Carlo checks. After solving, we reverse the output to recover $S(t)$ on the original time grid.

Since we need to evaluate $S(t)$ at many different times during training (including times not on the original grid), we build a cubic spline interpolant for each of the four entries $S_{ij}(t)$ using `scipy.interpolate.interp1d`. This lets us evaluate S cheaply at arbitrary $t \in [0, T]$.
We also need the integral correction term

$$b(t) = \int_t^T \operatorname{tr}\big(\sigma\sigma^\top S(r)\big)\, dr.$$

We compute this on the time grid using the trapezoidal rule, sweeping backwards from $b(T) = 0$, and store it as a further cubic spline interpolant. This is straightforward since we already have $S(r)$ evaluated on the grid.

Once the Riccati ODE has been solved, evaluating v and α^* is just a matter of plugging in. For a batch of times $t \in \mathbb{R}^{N_{\text{batch}}}$ and states $x \in \mathbb{R}^{N_{\text{batch}} \times 1 \times 2}$ given as `torch` tensors, we evaluate

$$v(t_i, x_i) = x_i^\top S(t_i)\, x_i + b(t_i)$$

by calling the spline interpolants on the numpy version of the time batch, then converting back to torch for the matrix multiplications. The output has shape $N_{\text{batch}} \times 1$. Similarly, the optimal control

$$\alpha^*(t_i, x_i) = -D^{-1}M^\top S(t_i)\, x_i$$

is computed by first evaluating $S(t_i)x_i$ via a batched matrix-vector product and then applying $-D^{-1}M^\top$, returning a tensor of shape $N_{\text{batch}} \times 2$. We precompute D^{-1} and $MD^{-1}M^\top$ at initialisation since these don't change.

We test the solver with a weak positive drift and modest noise level and $M = C = D = R = I_2$. Specifically

$$H = 0.1\, I_2, \quad M = I_2, \quad C = I_2, \quad D = I_2, \quad R = I_2, \quad \sigma = 0.3\, I_2, \quad T = 1$$

where $I_2 \in \mathbb{R}^{2\times 2}$ is the 2×2 identity matrix. The solver hits the terminal condition to machine precision ($|S(T) - R| = 0$). Figure 1 shows the four components of $S(t)$: the off-diagonal entries remain zero throughout due to the decoupled structure of the problem, and the diagonal entries increase monotonically from 1 at $t = T$ to ≈ 1.090 at $t = 0$ as expected. We also spot-check that $\alpha^*(0, (1, 1)^\top) \approx (-1.090, -1.090)^\top = -S(0)x$ and $\alpha^*(1, (0, 0)^\top) = (0, 0)^\top$, both consistent with the analytical formula.

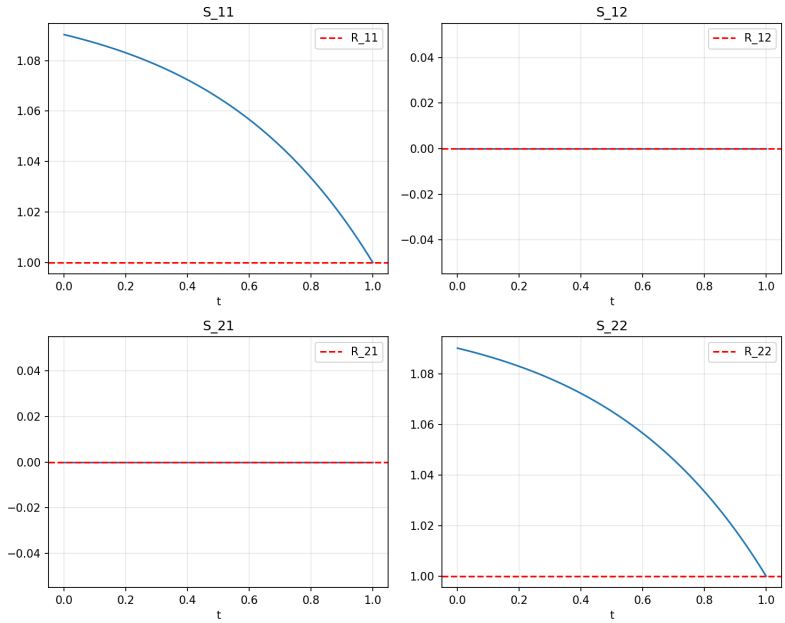

Figure 1: Components of the Riccati solution $S(t)$ on $[0, T]$. Dashed red lines show the terminal condition $S(T) = R = I_2$.

1.2 LQR Monte Carlo Checks

This section is devoted to verifying that the analytical solution we obtained in Exercise 1.1 is actually correct. The idea is to use the fact that the value function $v(t_0, x_0)$ is defined as the expected cost when starting at (t_0, x_0) and using the optimal control α^*. So if we actually simulate the system many times using the optimal control and average the cost we get, we should recover $v(t_0, x_0)$. If we don't, it is very likely that something is wrong with either our Riccati solver or our simulation.

More precisely, recall that the value function is

$$v(t_0, x_0) = \inf_{\alpha} \mathbb{E}^{t_0,x_0} \left[\int_{t_0}^{T} \big(X_s^\top C X_s + \alpha_s^\top D \alpha_s\big) ds + X_T^\top R X_T \right],$$

and the infimum is achieved by the optimal control $\alpha^*(t, x) = -D^{-1} M^\top S(t) x$. So substituting this in, we have

$$v(t_0, x_0) = \mathbb{E}^{t_0,x_0} \left[\int_{t_0}^{T} \big(X_s^\top C X_s + \alpha^*(s, X_s)^\top D \alpha^*(s, X_s)\big) ds + X_T^\top R X_T \right],$$

where X now follows the optimally controlled SDE. With α^* plugged in, this SDE becomes linear in X

$$dX_s = \big(H - M D^{-1} M^\top S(s)\big) X_s \, ds + \sigma \, dW_s.$$

We will use Monte Carlo to estimate the expectation above. We will do this by simulating N_{MC} independent realisations of this SDE, each driven by a different Brownian path, and averaging the realised cost over all of them. Concretely, for each simulated path we compute

$$\hat{J}^{(i)} = \int_{t_0}^{T} \big((X_s^{(i)})^\top C X_s^{(i)} + \alpha^*(s, X_s^{(i)})^\top D \alpha^*(s, X_s^{(i)})\big) ds + (X_T^{(i)})^\top R X_T^{(i)},$$

and our Monte Carlo estimate is $\hat{v} = \frac{1}{N_{\mathrm{MC}}} \sum_{i=1}^{N_{\mathrm{MC}}} \hat{J}^{(i)}$. By the law of large numbers, $\hat{v} \to v(t_0, x_0)$ as $N_{\mathrm{MC}} \to \infty$, almost surely.

We use the absolute error $|\hat{v} - v_{\mathrm{true}}|$ as our error measure, where v_{true} is the value from the Riccati solution. This is the natural choice here since we have the exact value available and we are

working with a single fixed starting point (t_0, x_0), so there is no need to normalise. We fix $t_0 = 0$, $x_0 = (1,1)^\top$ throughout, giving $v_{\text{true}} = v(0,(1,1)^\top) \approx 2.3711$.

Of course one cannot simulate a continuous-time SDE on a computer; instead, one must discretise it in time. We implement two schemes: explicit and implicit Euler. For a time grid $t_0 < t_1 < \cdots < t_N = T$ with uniform step size $\tau = T/N$, the explicit scheme approximates the SDE by

$$X_{t_{n+1}} = X_{t_n} + \tau\big(H - MD^{-1}M^\top S(t_n)\big)X_{t_n} + \sigma\Delta W_n,$$

where $\Delta W_n = W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, \tau I)$. This is the most straightforward discretisation: we just use everything evaluated at the current time t_n to step forward. The running cost is accumulated using X_{t_n} at the left endpoint of each interval.

The implicit scheme is slightly more involved. Instead of using $S(t_n)$ we use $S(t_{n+1})$, and critically the unknown $X_{t_{n+1}}$ appears on both sides

$$X_{t_{n+1}} = X_{t_n} + \tau\big(H - MD^{-1}M^\top S(t_{n+1})\big)X_{t_{n+1}} + \sigma\Delta W_n.$$

This might look circular, but since everything is linear in $X_{t_{n+1}}$ we can just collect the $X_{t_{n+1}}$ terms on the left-hand side to get

$$\big(I - \tau A_{n+1}\big)X_{t_{n+1}} = X_{t_n} + \sigma\Delta W_n, \qquad A_{n+1} = H - MD^{-1}M^\top S(t_{n+1}),$$

which is a 2×2 linear system that we can solve for $X_{t_{n+1}}$ directly. The reason for going to this extra effort is that implicit methods are generally more stable for large time steps - they tend to damp out large oscillations rather than amplify them, so we would expect the implicit scheme to perform better when τ is large. Importantly, since the system matrix $I - \tau A_{n+1}$ does not depend on the specific Brownian path, it is the same for all N_{MC} paths at a given time step. So we only need to solve it once per step and apply it to all paths simultaneously, which we do efficiently using `torch.linalg.solve`.

We run two separate convergence studies. In the first, we fix $N_{\text{MC}} = 10^5$ and vary $N \in \{1, 10, 50, 100, 500, 1000, 5000\}$. Since we are measuring error in an expected cost, the relevant result is the weak convergence rate of Euler-Maruyama, which is $O(\tau)$. We therefore expect a slope of 1 on a log-log plot, which Figure 2 confirms for both schemes approximatly when τ is greater than 10^{-2}. Below $\tau \approx 10^{-3}$ the error flattens somewhat. This flattening is not the Euler scheme failing - recall that v_{true} is itself a numerical approximation, so once τ is small enough the error is dominated by the Riccati solver rather than the simulation. We also noticed the implicit scheme performs no better than the explicit one, which makes sense since the optimal control already stabilises the SDE.

In the second study we fix $N = 5000$ and vary $N_{\text{MC}} \in \{10, 50, 10^2, 5 \times 10^2, 10^3, 5 \cdot 10^3, 10^4, 5 \cdot 10^4, 10^5\}$. By the CLT the standard error scales as $\sigma/\sqrt{N_{\text{MC}}}$, so we expect $O(1/\sqrt{N_{\text{MC}}})$ decay. Figure 3 confirms this cleanly for the standard error (orange). The absolute error (blue) is noisier and non-monotone, which is expected - for small N_{MC} each run tells you very little on its own. By $N_{\text{MC}} = 10^5$ the estimate $\hat{v} \approx 2.370$ agrees with $v_{\text{true}} = 2.371$ to three decimal places, confirming that both the Riccati solver and the simulation are correct.

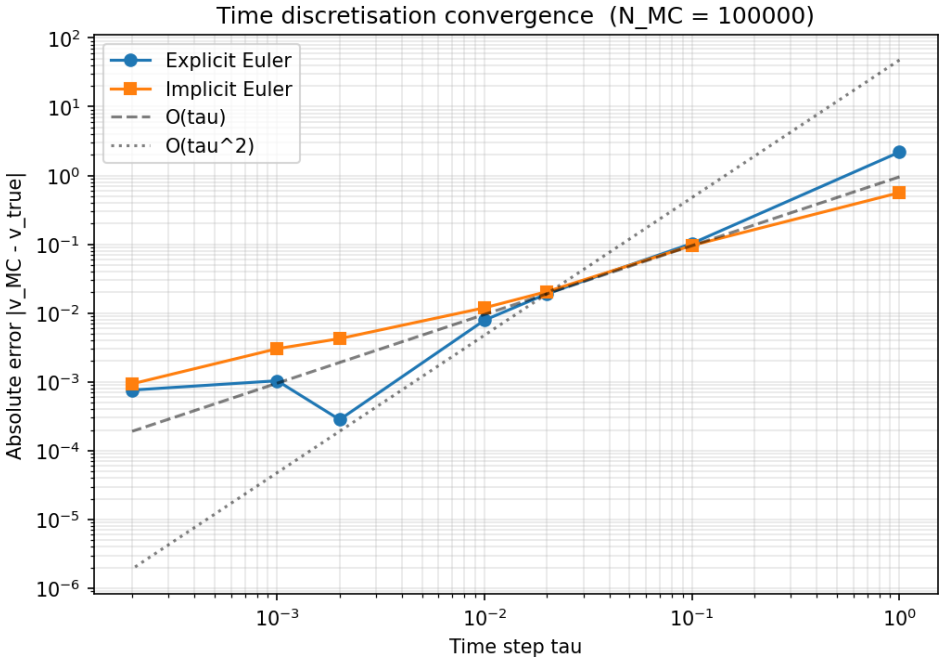

Figure 2: Time discretisation convergence with $N_{\mathrm{MC}} = 10^5$. Both schemes follow $O(\tau)$ for large τ before flattening at $\sim 10^{-3}$, which is the accuracy floor of the Riccati ODE solver rather than the Euler scheme.

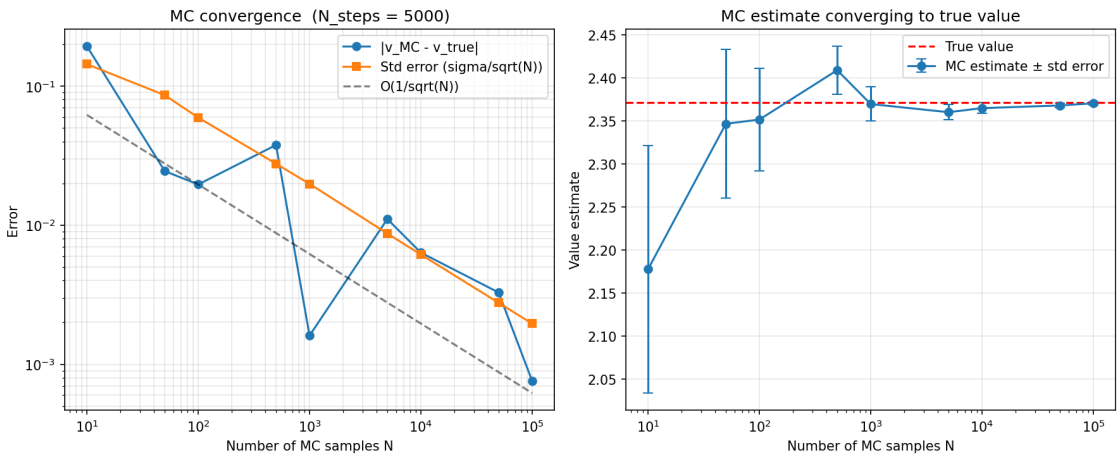

Figure 3: MC sample convergence with $N = 5000$ time steps. Left: the standard error follows $O(1/\sqrt{N})$ as predicted by the CLT; the absolute error is noisier but trends in the same direction. Right: MC estimates with error bars converging to the true value (dashed red) as N_{MC} increases.